BITS Pilani

Work Integrated Learning Programmes

SECURE SOFTWARE ENGINEERING

Lab Sheet: Secure Coding and Security Awareness

Course Title	Secure Software Engineering
Assignment Type	Lab Sheet
Total Marks	20
Expected Time	30 to 40 minutes (after the setup below)
Language	Python OR Java OR JavaScript (Node.js). Exercise 4 uses HTML/JavaScript.

General Instructions

- Work individually. Each exercise is independent.
- For Exercises 1 to 3, choose any ONE language and use it for all three.
- **NN = last two digits of your BITS ID. BITS_ID = your full BITS ID.**
- The first line of output of every program you run must print your BITS_ID. Screenshots without it get no marks.
- Use only your own machine, localhost and the dummy data given here. Do not test any other system.

Setup (do this before you start)

Choice	You need
Python	Python 3.9 or later. No extra packages.
Java	JDK 11 or later. For Exercise 2, the SQLite JDBC driver jar (org.xerial sqlite-jdbc) from Maven Central.
JavaScript	Node.js 22.13 or later (has built-in SQLite).
All	A browser, and Python or Node.js to serve one local web page in Exercise 4.

Required Folder and Run Log

• Create one folder for each exercise: ex1, ex2, ex3 and ex4. Keep the original starter file and the corrected file in the same folder.

• Save terminal output as run.txt. Windows example: python checkout.py > run.txt 2>&1. macOS/Linux example: python3 checkout.py > run.txt 2>&1. Use the equivalent java or node command for the selected language.

• At the end of run.txt, add the runtime version command used: python --version, java -version or node --version.

• ZIP the editable source files and run logs. Password-protected archives, image-only code and compiled files without source are not accepted.

Marks Distribution

Exercise	Topic	Marks
1	Input Validation at the Trust Boundary	4
2	SQL Injection and Parameterised Queries	4
3	Path Traversal and Safe Error Messages	4
4	XSS: Safe DOM Output and URL Checks	4

Exercise	Topic	Marks
5	Phishing Awareness	4
	TOTAL	**20**

Exercise 1: Input Validation at the Trust Boundary [4 Marks]

Background: The price and quantity in a checkout request come from the client and cannot be trusted. The server must use its own price and check the quantity before calculating the total.

Theory: A trust boundary is the point where data moves from a less trusted source, such as a browser or mobile application, into a trusted server component. Client-supplied values may be missing, malformed or deliberately changed. Secure validation checks both the data type and the business rule. In this exercise, quantity is client input, but product price is server-owned data and must not be accepted from the client.

Learning Objectives

- Identify data that crosses a trust boundary.
- Differentiate type validation from business-rule validation.
- Use server-owned values for security-sensitive calculations.
- Test valid, boundary and invalid inputs.

Starter code (choose one)

Python (`checkout.py`)

```
def checkout(server_price, qty, client_price):
    total = client_price * qty
    return {"qty": qty, "total": total}

print(checkout(55000, -2, 1))
```

Java (`Checkout.java`)

```
public class Checkout {
    static long checkout(long serverPrice, int qty, long clientPrice) {
        long total = clientPrice * qty;
        return total;
    }

    public static void main(String[] args) {
        System.out.println(checkout(55000, -2, 1));
    }
}
```

JavaScript (`checkout.js`)

```
function checkout(serverPrice, qty, clientPrice) {
  const total = clientPrice * qty;
  return { qty, total };
}

console.log(checkout(55000, -2, 1));
```

Tasks

1. Run the code unchanged. Note the total it prints.
2. Change the function so that the total uses **only the server price**, and the quantity must be a **whole number from 1 to 10**. Any other quantity must be rejected with an error message (do not silently correct it).
3. Call the corrected function with these quantities: **(NN mod 10) + 1**, 0, 11, -2 and 2.5. In Java, replace 2.5 with the string "2.5" read through `Integer.parseInt`, and handle the exception.

What to Submit

• Keep the original and corrected files. The evaluator may change the valid quantity range and ask for an immediate rerun.

- Corrected code.
- Screenshot of the output before and after the change.
- Two lines: why is trusting `client_price` a security problem and not just a bug?

Assessment criterion	Marks
Server price used; client price ignored	1.5

Assessment criterion	Marks
Quantity check correct for all five test values	1.5
Before/after output and the two-line answer	1.0
Total	**4**

Exercise 2: SQL Injection and Parameterised Queries [4 Marks]

Background: SQL injection happens when user input is joined into the SQL text. A parameterised query sends the value separately, so it is never read as SQL. A column name in ORDER BY cannot be a parameter, so it must come from a fixed allow-list.

Theory: SQL injection occurs when untrusted input is combined with SQL command text. A parameterised query keeps the SQL structure separate from data values, so input is treated as data rather than executable SQL. Parameters can represent values such as email addresses, but they cannot safely replace SQL identifiers such as column names. Dynamic identifiers therefore require a fixed allow-list and explicit mapping.

Learning Objectives

- Observe how concatenated input changes the meaning of an SQL query.
- Use a prepared or parameterised query for user-supplied values.
- Apply an allow-list when a query contains a selectable sort field.
- Verify that malicious input does not change database contents.

Starter code (choose one). Each program creates an in-memory database with six dummy users.

Python (users.py), run as python users.py <BITS_ID>

```
import sqlite3
import sys

BITS_ID = sys.argv[1]
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT, email TEXT)")
conn.executemany("INSERT INTO users (name, email) VALUES (?, ?)", [
    ("Asha", "asha@lab.test"), ("Ravi", "ravi@lab.test"),
    ("Meena", "meena@lab.test"), ("Kiran", "kiran@lab.test"),
    ("Arun", "arun@lab.test"), (BITS_ID, BITS_ID.lower() + "@lab.test"),
])


def find_user(email, sort):
    sql = "SELECT id, name, email FROM users WHERE email = '" + email + "' ORDER BY " + sort
    print("SQL:", sql)
    return conn.execute(sql).fetchall()


print(find_user("ravi@lab.test", "name"))
print(find_user("' OR '1'='1' --", "name"))
```

JavaScript (users.js), run as node users.js <BITS_ID>

```
const { DatabaseSync } = require("node:sqlite");

const BITS_ID = process.argv[2];
const db = new DatabaseSync(":memory:");
db.exec("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT, email TEXT)");
const ins = db.prepare("INSERT INTO users (name, email) VALUES (?, ?)");
[["Asha", "asha@lab.test"], ["Ravi", "ravi@lab.test"], ["Meena", "meena@lab.test"],
 ["Kiran", "kiran@lab.test"], ["Arun", "arun@lab.test"], [BITS_ID, BITS_ID.toLowerCase() +
"@lab.test"]]
  .forEach(([n, e]) => ins.run(n, e));

function findUser(email, sort) {
  const sql = "SELECT id, name, email FROM users WHERE email = '" + email + "' ORDER BY " +
sort;
  console.log("SQL:", sql);
  return db.prepare(sql).all();
}

console.log(findUser("ravi@lab.test", "name"));
console.log(findUser("' OR '1'='1' --", "name"));
```

Java (Users.java), run as java -cp .:sqlite-jdbc.jar Users <BITS_ID> (on Windows use ; instead of :)

```
import java.sql.*;

public class Users {
    static Connection conn;

    static void findUser(String email, String sort) throws SQLException {
        String sql = "SELECT id, name, email FROM users WHERE email = '" + email + "' ORDER
BY " + sort;
        System.out.println("SQL: " + sql);
        ResultSet rs = conn.createStatement().executeQuery(sql);
        while (rs.next()) {
            System.out.println(rs.getInt(1) + " " + rs.getString(2) + " " +
rs.getString(3));
        }
    }

    public static void main(String[] args) throws SQLException {
        String bitsId = args[0];
        conn = DriverManager.getConnection("jdbc:sqlite::memory:");
        conn.createStatement().execute("CREATE TABLE users (id INTEGER PRIMARY KEY, name
TEXT, email TEXT)");
        String[][] rows = {{"Asha", "asha@lab.test"}, {"Ravi", "ravi@lab.test"}, {"Meena",
"meena@lab.test"},
                {"Kiran", "kiran@lab.test"}, {"Arun", "arun@lab.test"}, {bitsId,
bitsId.toLowerCase() + "@lab.test"}};
        PreparedStatement ins = conn.prepareStatement("INSERT INTO users (name, email)
VALUES (?, ?)");
        for (String[] r : rows) {
            ins.setString(1, r[0]);
            ins.setString(2, r[1]);
            ins.executeUpdate();
        }
        findUser("ravi@lab.test", "name");
        findUser("' OR '1'='1' --", "name");
    }
}
```

Tasks

1. Run the program. Note the SQL printed for the second call and how many rows it returns.
2. Change `find_user` so the email is passed as a **bound parameter** (`?`), not joined into the string.
3. Accept only `NAME` or `EMAIL` as the sort value. Map them to the column names `name` and `email`. Reject anything else with an error. Add `LIMIT 10` to the query.
4. Run three calls: your own lab email (BITS_ID in lower case + @lab.test) sorted by NAME; the injection string ' OR '1'='1' --; and sort value name; DROP TABLE users. Then print SELECT COUNT(*) FROM users.

What to Submit

• Submit run.txt showing the valid query, injection test, rejected sort value and final row count in one execution. The evaluator may add one sort label or hostile email input.

- Corrected code.
- Screenshot of the output before and after the change. After the change, the injection call must return no rows, the bad sort must be rejected, and the count must still be 6.
- Two lines: why can the sort column not be passed as a `?` parameter?

Assessment criterion	Marks
Email passed as a bound parameter	1.5
Sort allow-list and LIMIT 10	1.0
Before/after output: injection returns no rows, bad sort rejected, count is 6	1.0
Two-line answer	0.5
Total	**4**

Exercise 3: Path Traversal and Safe Error Messages [4 Marks]

Background: A file name sent by a user is untrusted. Joining it to a folder name is not enough, because ../ can move out of that folder. The program must check the final resolved path and must not return raw error details to the user.

Theory: Path traversal occurs when input such as ../ causes file access to move outside the intended directory. Checking only the original string or joining a folder and filename is not sufficient. The application must resolve and normalise the final path, then verify that it remains inside the approved base directory. Error responses should be useful to the user without exposing stack traces, absolute paths or secrets.

Learning Objectives

- Recognise path traversal and unsafe file-access assumptions.
- Validate a resolved path against an approved base directory.
- Restrict file access by type and location.
- Return a generic error while recording a matching support reference.

Setup: create this folder by hand (any editor)

```
lab/
  secret.txt              contains: DB_PASSWORD=lab-only-secret
  reports/
    summary.txt           contains: Semester summary
    report_NN.txt         contains: Report for <BITS_ID>
    notes.csv             contains: a,b
```

Save your program inside `lab/` and run it from `lab/`.

Starter code (choose one)

Python (`reader.py`)

```
from pathlib import Path

BASE = Path("reports")


def read_report(name):
    try:
        return (BASE / name).read_text()
    except Exception as e:
        return str(e)


for name in ["summary.txt", "missing.txt", "../secret.txt"]:
    print(name, "->", read_report(name))
```

Java (`Reader.java`)

```
import java.nio.file.*;

public class Reader {
    static String readReport(String name) {
        try {
            return Files.readString(Paths.get("reports", name));
        } catch (Exception e) {
            return e.toString();
        }
    }

    public static void main(String[] args) {
        for (String name : new String[]{"summary.txt", "missing.txt", "../secret.txt"}) {
            System.out.println(name + " -> " + readReport(name));
        }
    }
}
```

JavaScript (`reader.js`)

```
const fs = require("fs");
const path = require("path");

function readReport(name) {
  try {
    return fs.readFileSync(path.join("reports", name), "utf8");
  } catch (e) {
    return e.stack;
  }
}

for (const name of ["summary.txt", "missing.txt", "../secret.txt"]) {
  console.log(name, "->", readReport(name));
}
```

Tasks

1. Run the program. Note what `../secret.txt` and `missing.txt` return.
2. Fix `read_report`: resolve the full path, and allow it only if it is **inside the resolved reports folder** and ends with **.txt**. Python: `Path.resolve()` and `is_relative_to()`. Java: `toAbsolutePath().normalize()` and `startsWith()` on Path. JavaScript: `path.resolve()` and check that the result starts with the reports folder path followed by `path.sep`.
3. For every failure, return only `Report not available (ref <id>)`, where `<id>` is a short random value. Print one log line to the console with the requested name and the same `<id>`, so a support person can match the two.
4. Test with: `summary.txt`, `report_NN.txt`, `missing.txt`, `../secret.txt` and `notes.csv`.

What to Submit

• Submit run.txt showing all five filenames in one execution. The evaluator may provide one additional traversal form or allowed extension.

- Corrected code.
- Screenshot of the output before and after the change.
- Two lines: why is a plain string check like `startsWith("reports")` on the joined path not enough?

Assessment criterion	Marks
Resolved-path check and .txt-only rule	1.5
Generic error with reference; no exception text or full path shown to the user	1.0
Before/after output for all five test names	1.0
Two-line answer	0.5
Total	**4**

Exercise 4: XSS: Safe DOM Output and URL Checks [4 Marks]

Background: innerHTML reads text as HTML, so user input can add markup or script to the page. textContent shows it as plain text. A link URL also needs a scheme check, because a javascript: URL runs code when clicked.

Theory: Cross-Site Scripting (XSS) occurs when untrusted data is written into a browser page as executable HTML or script. innerHTML parses a string as markup, whereas textContent displays the same value as text. URL values require separate validation because a link can contain an unsafe scheme such as javascript:. Output handling must therefore match the browser context in which the value is used.

Learning Objectives

- Observe the difference between HTML interpretation and plain-text output.
- Use textContent for untrusted text displayed in the DOM.
- Parse a URL and allow only approved protocols.
- Explain how Content Security Policy provides an additional layer of protection.

Starter code: save as `profile.html`

```
<!doctype html>
<html>
<body>
  <h2 id="welcome"></h2>
  <a id="site">Open profile website</a>
  <script>
    const p = new URLSearchParams(location.search);
    const name = p.get("name") || "Student";
    const website = p.get("website") || "https://example.com";
    document.getElementById("welcome").innerHTML = "Welcome " + name;
    document.getElementById("site").setAttribute("href", website);
  </script>
</body>
</html>
```

Tasks

1. In the folder with `profile.html`, start a local server on port **90NN**: `python -m http.server 90NN` or `npx http-server -p 90NN`.
2. Open http://localhost:90NN/profile.html?name=BITS_ID. Note that your BITS_ID appears in bold, which shows that the input was read as HTML.
3. Open `http://localhost:90NN/profile.html?website=javascript:alert('NN')` and click the link. Note what happens.
4. Fix the page: use `textContent` for the name. Check the website with `new URL(website)` inside try/catch, and set the link only if the protocol is `http:` or `https:`. Otherwise set `href` to `#`.
5. Repeat steps 2 and 3, and also test `website=https://example.com` and `website=abc`.

What to Submit

• Submit the editable profile.html. The evaluator may provide one additional URL scheme or HTML-like name and ask the student to predict and verify the result.

- Corrected `profile.html`.
- Screenshots of the four tests after the fix (address bar visible). For the link tests, show the `href` in browser developer tools (Elements tab).
- Two lines: a Content-Security-Policy of `script-src 'self'` would have blocked the starter page's own script. Why?

Assessment criterion	Marks
textContent used; markup shown as text	1.0
URL check allows only http and https; bad input handled	1.5
Screenshots of the four tests with href visible	1.0
Two-line answer	0.5
Total	**4**

Exercise 5: Phishing Awareness [4 Marks]

Background: Phishing remains one of the most common ways attackers get in. Most phishing emails can be spotted from a few warning signs if you know where to look.

Theory: Phishing attempts to influence a person into revealing information, opening a link or performing an unsafe action. Common indicators include a misleading sender domain, urgency, threats, unusual requests and deceptive links. The visible words in a link may resemble a trusted service while the registered domain belongs to someone else. Verification should use a known official channel rather than contact details supplied in the suspicious message.

Learning Objectives

- Identify phishing indicators in a realistic email example.
- Determine the effective domain in a deceptive link.
- Distinguish message claims from verifiable evidence.
- Select safe verification steps without opening the supplied link.

Part A: Phishing Quiz (1.5 Marks)

1. Open the Google Phishing Quiz: https://phishingquiz.withgoogle.com/
2. Use your BITS ID when the quiz asks for a name.
3. Complete all questions and take a screenshot of the final score page.

Part B: Find the Red Flags (2.5 Marks)

Read the email below. It is a made-up training example.

```
From:    BITS WILP Exam Cell <examcell@bits-wilp-results.net>
To:      undisclosed-recipients
Subject: URGENT: Your exam result is ON HOLD - verify in 24 hours

Dear Student,

Due to a fee mismatch your Mid-Semester result has been withheld.
To avoid cancelation of your registration, verify your eLearn login
within 24 hours using the secure link below:

    https://elearn.bits-pilani.ac.in.verify-login.co/results

You will also need to confirm your bank card for a refund of Rs 1,250.

Regards,
Exam Cell Team
BITS Pilani (WILP)
```

1. List **five** red flags. For each one, quote the exact text from the email that shows it.
2. Write the real domain in the link above. Explain in one line how you worked it out.
3. Write two steps you would take to check whether such an email is genuine, without clicking anything in it.

What to Submit

• Part B answers must quote the given email. Generic lists of phishing signs without evidence from the email receive no marks.

- Screenshot of the quiz score page showing your BITS_ID.
- Answers to Part B questions 1 to 3.

Assessment criterion	Marks
Quiz score screenshot with BITS_ID	1.5
Five red flags, each with the quoted text	1.5
Real domain identified with reason	0.5
Two sensible verification steps	0.5
Total	**4**